

과목명: 프로그래밍언어

개발 및 테스트 환경: Windows / WSL2 Ubuntu 24.04.3 LTS / g++ 13.3.0 / Python 3.12.0

C++ compile: g++ -std=c++17 interpreter.cpp

C++ run: ./a.out

Python run: python interpreter.py

1. 구현 개요

본 과제에서는 주어진 EBNF 문법을 해석하는 인터프리터를 C++와 Python 으로 각각 구현하였다. 프로그램은 입력된 한 줄의 코드를 하나의 독립된 프로그램으로 처리하며, 빈 줄이 입력되면 종료된다.

입력 코드는 먼저 토큰 단위로 분리한 뒤, Recursive-Descent Parser 방식으로 EBNF 의 각 비단말 기호에 대응되는 함수를 호출하여 문법을 검사한다. 토큰라이저는 공백뿐 아니라 연산자, 괄호,세미콜론을 기준으로도 분리하며 두 자리 연산자(==, !=, <=, >=)를 한 단위로 인식하도록 구현하였다. 문법 검사가 성공하면 구성된 AST 를 순서대로 실행하고, 문법 오류가 있으면 "Syntax Error!"를 출력한다.

2. 구현 내용

문법의 각 비단말 기호에 대응되도록 program, declaration, statement, aexpr, term, bexpr 파싱 함수를 구성하였다. declaration 은 variable 과 constant 를 처리하며, variable 은 0 으로 자동 초기화하고 constant 는 선언 시 지정된 값을 저장한 뒤 이후 대입할 수 없도록 하였다. 변수/상수의 재선언, 키워드와 동일한 이름의 변수, 대문자 또는 숫자로 시작하는 이름은 모두 Syntax Error 로 거부한다.

statement 는 대입문, print 문, repeat-until 문, select-else 문으로 나누어 처리하였다. print 문과 대입문에는 산술식만 허용하고, 불리언식은 repeat-until 과 select-else 의 조건식에서만 사용되도록 하였다. EBNF 에 따라 선언이 모두 끝난 뒤에 문장이 등장해야 하므로, 문장 영역에서 variable 이나 constant 토큰이 다시 등장하면 Syntax Error 로 처리한다.

산술식은 과제 조건에 맞게 +, -, *를 모두 같은 우선순위로 보고 왼쪽 결합성으로 계산하였다. 괄호가 있는 경우에는 괄호 안의 식을 먼저 계산하도록 하였고, term 앞에 한 번의 단항 음수만 허용하여 - - x 같은 입력은 거부한다. constant 선언의 초기값은 EBNF 상 number 만 허용되므로 음수 초기값 역시 Syntax Error 로 거부한다.

repeat-until 문은 블록을 먼저 실행한 뒤 조건을 검사하도록 구현하였다. 명세서 예시에 맞게 조건이 참이면 반복을 계속하고, 거짓이면 종료한다. select-else 문은 조건이 참이면 첫 번째 블록, 거짓이면 else 블록을 실행한다.

출력 결과는 print 문을 만날 때 바로 출력하지 않고 임시 배열에 저장하였다. 한 입력 코드 안에서 뒤쪽에 오류가 발생하면 앞의 출력도 보여주지 않고 "Syntax Error!"만 출력해야 하기 때문이다. 또한 명세서에서 와 의 최대 길이를 10 자로 한정해도 된다고 하였으므로, 11 자 이상의 식별자나 숫자는 가정을 벗어난 입력으로 보아 Syntax Error 로 처리하였다.

추가로, 명세서 PDF 예제 일부에 일반 하이픈(-)이 아닌 En-dash(–)가 포함되어 있어 입력 문자열에서 이를 일반 하이픈으로 변환하는 전처리를 추가하였다. Python 구현에서는 숫자 판별을 ASCII 0~9 범위로 명시적으로 제한하여 유니코드 숫자 문자가 들어오더라도 C++ 구현과 동일하게 거부되도록 하였고, 깊은 괄호 중첩 입력을 대비해 재귀 한도를 충분히 늘려 두었다. C++ 구현에서는 줄 끝의 CR 문자를 제거하여 Windows 에서 작성된 입력 파일도 안정적으로 처리한다.

3. 실행 결과

C++와 Python 두 구현 모두 과제 명세서의 실행 예제를 입력하여 테스트하였다. 오류가 있는 입력은 "Syntax Error!"를 출력하였고, 정상 입력은 명세서 예시와 동일한 결과를 출력하였다. 아래는 Python 실행 결과와 C++ 실행 결과 스냅샷이다.

Terminal Local × + ▾

PS C:\Users\akftj\Desktop\PL_assign> python interpreter.py
abc = 10 ;
Syntax Error!
int b ;
Syntax Error!
variable k = 2 ;
Syntax Error!
constant pi = 314 ; pi = 100 ;
Syntax Error!
variable k ; variable j ; k = 3 ; j = 20 ; variable a ; print k + j ;
Syntax Error!
variable k ; variable j ; k = 3 ; j = 20 ; print k >= j ;
Syntax Error!
variable abc ; abc = 365 ;
variable x ; x = 100 ; print x ;

Terminal Local × + ▾

PS C:\Users\akftj\Desktop\PL_assign> wsl bash -lc "cd /mnt/c/Users/akftj/Desktop/PL_assign && g++ -std=c++17 interpreter.cpp && ./a.out"
abc = 10 ;
Syntax Error!
int b ;
Syntax Error!
variable k = 2 ;
Syntax Error!
constant pi = 314 ; pi = 100 ;
Syntax Error!
variable k ; variable j ; k = 3 ; j = 20 ; variable a ; print k + j ;
Syntax Error!
variable k ; variable j ; k = 3 ; j = 20 ; print k >= j ;
Syntax Error!
variable abc ; abc = 365 ;
variable x ; x = 100 ; print x ;
100
variable k ; variable j ; constant n = 5 ; k = 3 ; j = 20 ; print k + j ; print n ;
23 5
variable k ; print k + 100 * 3 - 1 ;
299
variable x ; x = 10 + 5 - (2 + 3 - 5 * 10) ; print x ;
15
variable x ; variable y ; x = 10 + 5 * 2 ; y = 20 - 5 ; select (x >= y) begin print x - 5 ; end else begin x = 7 ; print x + 5 ; end ;
25
variable x ; variable y ; x = 10 + 5 * 2 ; y = 20 - 5 ; select (x <= y) begin print x - 5 ; end else begin x = 7 ; print x + 5 ; end ;
12
variable k ; variable j ; k = 3 ; j = 20 ; repeat begin print k ; k = k + 1 ; end until (k <= 5) ;
3 4 5
variable k ; variable j ; k = 30 ; j = 25 ; repeat begin print k - j * 10 ; k = k - 1 ; end until (k > j) ; print k ;
50 40 30 20 10 25
variable i ; variable j ; variable k ; i = 0 ; j = 5 ; k = 3 ; repeat begin select (i < k) begin i = i + 1 ; print i ; end else begin i = i + 1 ; end ; end until (i < j) ; print i ; print j * k ;
1 2 3 5 15
PS C:\Users\akftj\Desktop\PL_assign>